



Instituto Tecnológico de Costa Rica

Campus Tecnológico Central Cartago

Escuela de Ingeniería en Computación

Curso IC-1803 Taller de programación

Proyecto 3 - Progranator

Estudiante:

Gabriel Montero Garro – 2026000288

Profesor: Diego Mora Rojas

Fecha de entrega: 26/06/2026

Periodo: I semestre 2026

Contenido

Introducción	3
Clase Nodo	3
Atributos principales	3
Métodos principales	3
Clase Arbol	4
Atributos principales	4
Métodos principales	4
Clase AppGUI	5
Atributos principales	5
Métodos principales	5
Relación entre las clases	6
Representación del árbol	6
Recorrido del árbol	6
Agregar una nueva pregunta y respuesta	7
Guardado del árbol.....	7
Carga del árbol desde archivo.....	7
Conclusión	7

Introducción

En este documento se explica el diseño orientado a objetos utilizado en el desarrollo de **PROGRANATOR**. El programa fue dividido en varias clases para separar la lógica del juego de la interfaz gráfica y facilitar la organización del código. Además, se describe cómo se representa el árbol de decisión, cómo se recorre durante una partida, cómo el programa aprende nuevas respuestas y cómo guarda y carga la información utilizando archivos JSON.

Clase Nodo

La clase **Nodo** representa cada elemento del árbol de decisión. Cada nodo puede almacenar una pregunta o una respuesta final. Además mantiene las referencias hacia los hijos correspondientes a las respuestas **Sí** y **No**.

Atributos principales

- **contenido:** almacena el texto de la pregunta o de la respuesta.
- **si:** referencia al hijo que se recorre cuando la respuesta es "Sí".
- **no:** referencia al hijo que se recorre cuando la respuesta es "No".

Métodos principales

es_hoja()

Verifica si el nodo corresponde a una respuesta final. Esto ocurre cuando no tiene hijos.

a_diccionario()

Convierte el nodo y todos sus descendientes en un diccionario para poder guardarlos en un archivo JSON.

desde_diccionario()

Reconstruye un nodo utilizando un diccionario leído desde un archivo JSON y recupera toda la estructura del árbol.

Clase Arbol

La clase **Arbol** contiene toda la lógica del juego. Se encarga de recorrer el árbol, procesar las respuestas del usuario, aprender nuevas preguntas y respuestas, además de guardar y cargar el árbol desde archivos JSON.

Atributos principales

- **raiz:** almacena el primer nodo del árbol.
- **nodo_actual:** indica el nodo donde se encuentra la partida en ese momento.
- **ruta_archivo:** guarda la ubicación del archivo JSON utilizado.

Métodos principales

crear_arbol_default()

Crea el árbol inicial con una pregunta y dos respuestas básicas cuando todavía no existe un archivo guardado.

reiniciar_partida()

Regresa el recorrido del árbol a la raíz para iniciar una nueva partida.

es_pregunta_actual()

Indica si el nodo actual corresponde a una pregunta o a una respuesta final.

obtener_contenido_actual()

Devuelve el texto almacenado en el nodo actual para mostrarlo en la interfaz.

responder()

Recibe la respuesta del usuario y avanza al siguiente nodo según haya respondido "Sí" o "No".

aprender()

Cuando el programa no logra adivinar correctamente, reemplaza la respuesta incorrecta por una nueva pregunta y agrega dos nuevas respuestas al árbol.

guardar_json()

Guarda el árbol completo en un archivo JSON.

cargar_json()

Carga un árbol desde un archivo JSON y valida que su contenido sea correcto.

Clase AppGUI

La clase **AppGUI** administra toda la interfaz gráfica utilizando Tkinter. Se encarga de mostrar las diferentes pantallas, recibir las acciones del usuario y comunicarse con la clase Arbol para ejecutar la lógica del juego.

Atributos principales

- **root**: ventana principal de Tkinter.
- **arbol**: objeto de la clase Arbol que controla la lógica del juego.
- **canvas**: superficie donde se dibujan las pantallas.
- **btn_menu**: botón que permite regresar al menú principal.

Métodos principales

mostrar_pantalla_inicial()

Muestra el menú principal del programa.

iniciar_partida()

Reinicia el árbol y comienza una nueva partida.

mostrar_pregunta()

Presenta la pregunta actual y muestra los botones para responder.

responder_si() y responder_no()

Envían la respuesta del usuario al árbol y continúan el recorrido.

mostrar_resultado_adivinanza()

Pregunta al usuario si el programa adivinó correctamente.

mostrar_formulario_aprendizaje()

Solicita la nueva respuesta y la nueva pregunta cuando el programa falla.

procesar_aprendizaje()

Actualiza el árbol con la nueva información y guarda los cambios en el archivo JSON.

Relación entre las clases

Las clases trabajan de forma independiente, pero se comunican entre sí para que el programa funcione correctamente.

La clase **AppGUI** crea un objeto de la clase **Arbol** y utiliza sus métodos para controlar el desarrollo del juego.

La clase **Arbol** administra toda la lógica utilizando objetos de la clase **Nodo**, que son los encargados de formar el árbol de decisión.

La relación entre las clases puede representarse de la siguiente forma:

AppGUI → Arbol → Nodo

Representación del árbol

El conocimiento del programa se almacena mediante un árbol binario de decisión.

Cada nodo de pregunta tiene dos hijos:

- El hijo **si** representa el camino cuando el usuario responde "Sí".
- El hijo **no** representa el camino cuando el usuario responde "No".

Las hojas del árbol representan las respuestas finales que el programa intenta adivinar.

Recorrido del árbol

Cada partida comienza en la raíz del árbol.

Mientras el nodo actual corresponda a una pregunta, el usuario responde "Sí" o "No". Según la respuesta, el programa avanza al hijo correspondiente.

Cuando se llega a una hoja, el programa realiza su intento de adivinanza utilizando la respuesta almacenada en ese nodo.

Agregar una nueva pregunta y respuesta

Si el programa no logra adivinar correctamente, solicita al usuario la respuesta correcta, una nueva pregunta que permita diferenciar ambas respuestas y cuál de las dos corresponde al camino de "Sí".

Con esa información, la respuesta incorrecta se convierte en una nueva pregunta y se crean dos nuevos nodos: uno con la respuesta correcta y otro con la respuesta anterior.

Así el árbol aumenta de tamaño y el programa puede utilizar ese nuevo conocimiento en las siguientes partidas.

Guardado del árbol

El árbol se guarda en un archivo con formato JSON.

Primero, cada nodo se convierte en un diccionario mediante el método **a_diccionario()**. Después, la clase **Arbol** utiliza la librería json para escribir esa información en el archivo correspondiente.

Carga del árbol desde archivo

Cuando el usuario selecciona un archivo, la clase **Arbol** verifica que exista, que no esté vacío y que su contenido sea válido.

Luego utiliza el método **desde_diccionario()** para reconstruir toda la estructura del árbol y continuar utilizando la información almacenada.

Si ocurre algún error durante la carga, el programa muestra un mensaje al usuario y mantiene el árbol que ya estaba cargado.

Conclusión

El uso de programación orientada a objetos permitió dividir el proyecto en clases con responsabilidades bien definidas. Gracias a esta organización, la lógica del juego, la representación del árbol y la interfaz gráfica permanecen separadas, lo que hace que el código sea más fácil de entender y mantener. Además, el programa puede aprender nuevas respuestas y conservar esa información mediante archivos JSON, permitiendo que el árbol de decisión crezca con cada partida.